

Form Validation & Regular Expressions

HTML validation attributes, the submit event, and using regex to check exactly what the user typed.

HTML Validation

submit Event

Regex

.test()

1 What is Form Validation?

Form validation checks whether the data entered by the user is valid before the form is submitted.

Name should have enough characters.
Email should have a valid format.
Password should satisfy certain requirements.
Required fields should not be empty.

Validation can be done using:

1. HTML validation

Built-in browser attributes like required, pattern.

2. JavaScript validation

Custom logic on the submit event.

3. Regex

Pattern matching for precise format checks.

2 HTML Form Validation

HTML provides built-in validation attributes.

required

Makes a field compulsory.

```
<input required type="text">
```

The form cannot be submitted if the field is empty.

minlength

Sets the minimum number of characters.

```
<input minlength="3" type="text">
```

maxlength

Sets the maximum number of characters.

```
<input maxlength="10" type="text">
```

pattern

Allows a **regular expression** to define what input is valid.

```
<input required pattern="[a-z]{3,8}" type="text">
```

This allows lowercase letters with **3–8 characters**.

⚠ IMPORTANT

HTML validation is useful, but JavaScript/server-side validation is still important because client-side validation can be bypassed.

3 Form submit Event

The `submit` event runs when the user submits a form.

```
let form = document.querySelector("form");

form.addEventListener("submit", function (details) {
  // validation
});
```

4 preventDefault()

Normally, submitting a form causes the browser to perform its default form-submission behavior.

```
details.preventDefault();
```

This **stops the default behavior** so JavaScript can validate/process the data first.

Syntax

```
event.preventDefault();
```

5 Getting Input Value

```
let nm = document.querySelector("#name");

console.log(nm.value);
```

`.value` gives the text entered into an input.

Checking length

```
nm.value.length
```

`.length` gives the number of characters. Example:

```
if (nm.value.length <= 2) {  
  // invalid  
}
```

6 Showing Validation Errors

HTML:

```
<div id="hide">  
  <small>Your name should be greater than 2 characters</small>  
</div>
```

CSS:

```
#hide {  
  display: none;  
}
```

JavaScript:

```
if (nm.value.length <= 2) {  
  document.querySelector("#hide").style.display = "initial";  
} else {  
  document.querySelector("#hide").style.display = "none";  
}
```

Logic

<code><= 2</code>	→ show error
<code>> 2</code>	→ hide error

7 Complete Basic Form Validation

```
let nm = document.querySelector("#name");  
let form = document.querySelector("form");  
  
form.addEventListener("submit", function (details) {  
  details.preventDefault();  
  
  if (nm.value.length <= 2) {  
    document.querySelector("#hide").style.display = "initial";  
  } else {  
    document.querySelector("#hide").style.display = "none";  
  }  
});
```

REGULAR EXPRESSIONS (REGEX)

8 What is Regex?

Regex (Regular Expression) is a pattern used to search, match, or validate text.

Common uses:

Email validation · Name validation · Password validation · Phone number validation · Finding specific patterns in text

9 Creating a Regex

Regex can be written between `/ /`.

```
const regex = /pattern/;
```

Example:

```
const regex = /^[a-z]+$/;
```

10 .test()

The `.test()` method checks whether a string matches the regex. It returns:

<code>true</code>	→ match found
<code>false</code>	→ no match

```
const regex = /^[a-z]+$/;  
console.log(regex.test("harsh"));
```

Output: `true`

Example:

```
const regex = /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/;  
let ans = regex.test("harsh@test.com");  
console.log(ans);
```

Output: `true`

11 Email Validation Regex

A commonly used basic email regex:

```
/^[^\s@]+@[^\s@]+\.[^\s@]+$
```

Breakdown

^

Start of the string.

[^\s@]+

One or more characters that are not whitespace or @.

@

Requires an @.

[^\s@]+

One or more characters after @.

\.

Requires a literal dot.

[^\s@]+

One or more characters after the dot.

\$

End of the string.

So **harsh@test.com** matches the pattern.

NOTE

This is a basic validation regex, not a complete implementation of every valid email format.

12 Name Validation Regex

Example:

```
/^[a-zA-Z]{3,8}$/
```

This allows: only letters · minimum 3 characters · maximum 8 characters.

Harsh	✓
Abrar	✓
ABRAR	✓
Ab	✗
AbrarJoo	✓
AbrarJooX	✗

For lowercase letters only:

```
/^[a-z]{3,8}$/
```

13 Important Regex Symbols

Symbol	Meaning
<code>^</code>	Start of string
<code>\$</code>	End of string
<code>.</code>	Any character
<code>\.</code>	Literal <code>.</code>
<code>+</code>	One or more
<code>*</code>	Zero or more
<code>?</code>	Zero or one
<code>{3}</code>	Exactly 3
<code>{3,8}</code>	3 to 8
<code>[a-z]</code>	Lowercase letters
<code>[A-Z]</code>	Uppercase letters
<code>[0-9]</code>	Digits
<code>\d</code>	Digit
<code>\s</code>	Whitespace
<code>\S</code>	Non-whitespace
<code>\w</code>	Word character
<code>\W</code>	Non-word character
<code>[^...]</code>	NOT characters inside

14 Common Regex Examples

Only lowercase letters

```
/^[a-z]+$ /
```

Only letters

```
/^[a-zA-Z]+$ /
```

Letters, 3–8 characters

```
/^[a-zA-Z]{3,8}$ /
```

Only numbers

```
/^[0-9]+$ / or /^\d+$ /
```

10-digit number

```
/^\d{10}$ /
```

Basic email

```
/^[^\s@]+@[^\s@]+\.[^\s@]+$ /
```

15 Form Validation + Regex Together

```

let form = document.querySelector("form");
let name = document.querySelector("#name");

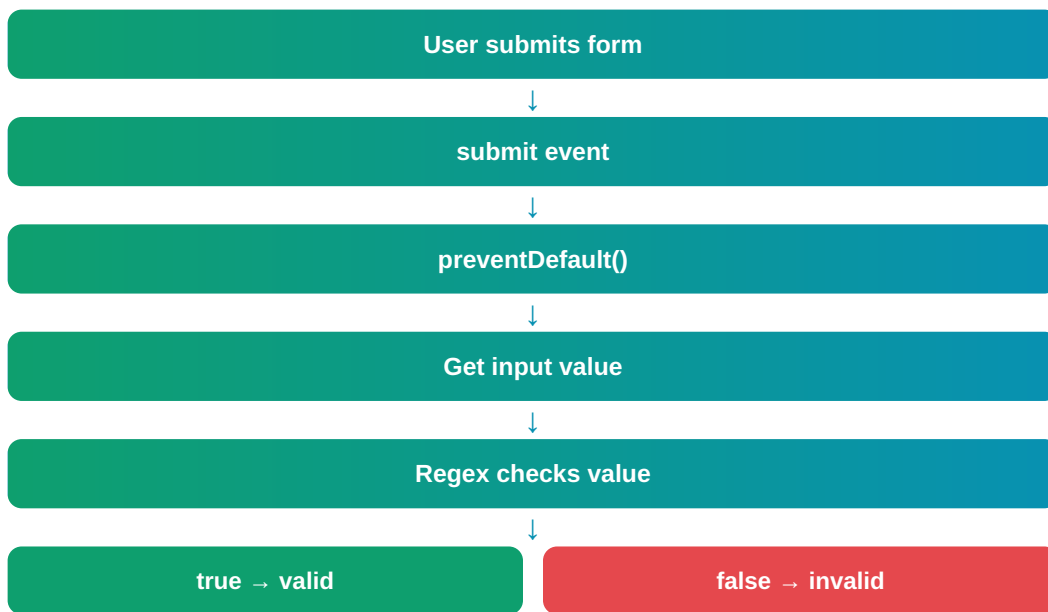
form.addEventListener("submit", function (details) {
  details.preventDefault();

  const regex = /^[a-zA-Z]{3,8}$/;

  if (!regex.test(name.value)) {
    console.log("Invalid name");
  } else {
    console.log("Valid name");
  }
});

```

Flow



BONUS — COMPLETE WORKING EXAMPLE

Putting everything together: an actual runnable form that validates the name field with regex and toggles the error message, styled and wired end-to-end.

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Document</title>
    <style>
      * { margin: 0; padding: 0; box-sizing: border-box; font-family: sans-serif; }
      body { padding: 40px; background: #f4f4f4; }
      input { display: block; padding: 8px; margin-bottom: 6px; width: 220px; }
      #hide { display: none; color: #c0392b; margin-bottom: 10px; }
      button { padding: 8px 16px; }
    </style>
  </head>
  <body>
    <form>
      <input id="name" type="text" placeholder="Name" />
      <div id="hide"><small>Your name should be greater than 2 characters</small></div>
      <button type="submit">Submit</button>
    </form>
  </body>
</html>

```

```
</form>
<script src="validate.js"></script>
</body>
</html>
```

```
let form = document.querySelector("form");
let nm = document.querySelector("#name");

const nameRegex = /^[a-zA-Z]{3,8}$/;

form.addEventListener("submit", function (details) {
  details.preventDefault();

  if (!nameRegex.test(nm.value)) {
    document.querySelector("#hide").style.display = "initial";
  } else {
    document.querySelector("#hide").style.display = "none";
    console.log("Form submitted:", nm.value);
  }
});
```



TIP

This combines sections 6, 7 and 15: the hidden error **div** , **preventDefault()** , and a regex check run together in one submit handler.

Key Things to Remember

- **required** → field cannot be empty.
- **minlength** → minimum characters.
- **maxlength** → maximum characters.
- **pattern** → HTML regex validation.
- **submit** → detects form submission.
- **preventDefault()** → stops default form submission.
- **.value** → gets input value.
- **.length** → gets character count.
- **Regex** → pattern matching.
- **.test()** → checks whether a string matches a regex.
- Regex returns **true** or **false**.
- Client-side validation can be bypassed, so important validation should also happen on the **server**.

🌟 HTML gives you quick built-in checks — regex gives you precise control over exactly what counts as valid.